SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – SCENARIO BASED

SAMPLE SOLUTION (REFERENCE FOR STUDENTS)

Course Code	**DSA0304**	**Course Title**	**Natural Language Processing**
CO Assessed	CO2	Assessment	Assessment Tool 2 – Scenario Based
Weightage	35% of CIA	Duration	30 minutes

Name: M S GOWRISH SHANKAR REDDY

Reg No:192424270

Annexure A – SCENARIO BASED

General method used in all questions

- Maintain a small lexicon of known root words.
- Define suffix and prefix rules in the form (affix, text to restore, affix type, word-class change).
- Strip the affix from the word, restore the root, and accept the result only if it is in the lexicon.
- Inflectional affixes change only the grammatical form (tense, number, aspect) and keep the word class. Derivational affixes create a new word, usually with a new meaning or word class.
- Map every variant to one normalized base form so that all of them match the same index entry.

Question 1: Morphological Processing for Search Engine Indexing

Scenario: A search engine receives the variants “analyzing”, “analysis” and “analytical”. They must be grouped under one representation to improve retrieval.

(a) Approach

1. Keep the root “analyze” in the lexicon.
2. Add rules: -ing (inflectional, restore e), -ysis → -yze (derivational), -ytical → -yze (derivational).
3. Strip the matching suffix, restore the root and verify it in the lexicon.
4. Classify each word as inflectional or derivational and record the word-class change.
5. Use the root “analyze” as the index key for all three words.

(b) Morphological Analysis

Word	Root	Affix(es)	Type	Class change	Normalized
analyzing	analyze	-ing	Inflectional	Verb → Verb (participle)	analyze
analysis	analyze	-sis	Derivational	Verb → Noun	analyze
analytical	analyze	-ic + -al	Derivational	Verb → Adjective	analyze

Morphological structure

- analyzing = analyze + -ing
- analysis = analy(ze) → analysis (noun formed with the Greek-origin suffix -sis; the stem changes from -yze to -ysis)
- analytical = analyze → analytic (-ic) → analytical (-al)

(c) Python Program

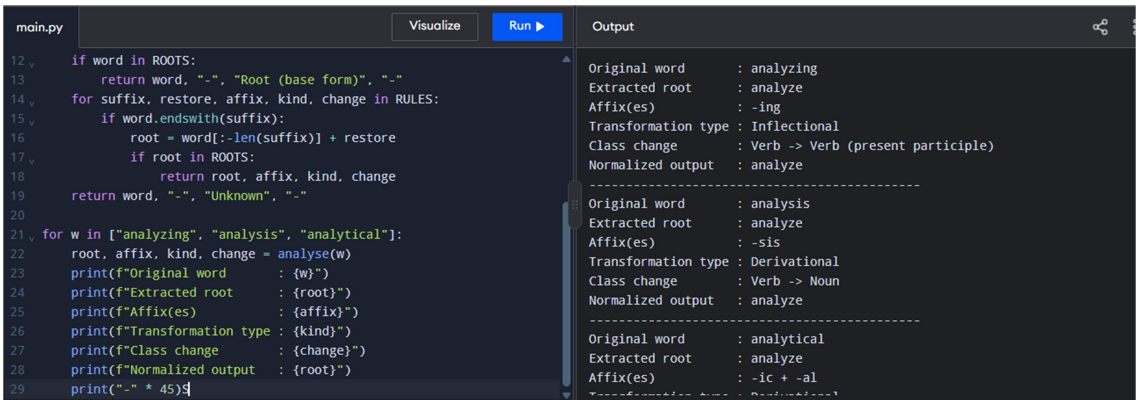
```
# Q1: Rule-based morphological processing for search-engine indexing

ROOTS = {"analyze"}          # known root words (mini lexicon)

# (suffix, text added back to recover root, affixes, type, class change)
RULES = [
    ("ytical", "yze", "-ic + -al", "Derivational", "Verb -> Adjective"),
    ("ysis",   "yze", "-sis",      "Derivational", "Verb -> Noun"),
    ("ing",    "e",   "-ing",      "Inflectional", "Verb -> Verb (present participle)"),
]

def analyse(word):
    if word in ROOTS:
        return word, "-", "Root (base form)", "-"
    for suffix, restore, affix, kind, change in RULES:
        if word.endswith(suffix):
            root = word[:-len(suffix)] + restore
            if root in ROOTS:
                return root, affix, kind, change
    return word, "-", "Unknown", "-"

for w in ["analyzing", "analysis", "analytical"]:
    root, affix, kind, change = analyse(w)
    print(f"Original word       : {w}")
    print(f"Extracted root      : {root}")
    print(f"Affix(es)           : {affix}")
    print(f"Transformation type : {kind}")
    print(f"Class change        : {change}")
    print(f"Normalized output   : {root}")
    print("-" * 45)
```



(d) Output

Original word : analyzing
Extracted root : analyze
Affix(es) : -ing
Transformation type : Inflectional
Class change : Verb -> Verb (present participle)
Normalized output : analyze

Original word : analysis
Extracted root : analyze
Affix(es) : -sis
Transformation type : Derivational
Class change : Verb -> Noun
Normalized output : analyze

Original word : analytical
Extracted root : analyze
Affix(es) : -ic + -al
Transformation type : Derivational
Class change : Verb -> Adjective
Normalized output : analyze

Question 2: Morphological Parsing for Sentiment Analysis

Scenario: Product reviews contain “disagree”, “agreement” and “agreeable”. The sentiment system must interpret them consistently.

(a) Approach

1. Keep the root “agree” in the lexicon.
2. Add the prefix rule dis- (negation) and the suffix rules -ment (Verb → Noun) and -able (Verb → Adjective).
3. Detect the prefix first, then the suffix; accept a split only if the remainder is a known root.
4. Record the meaning change caused by each affix.

5. Normalize all forms to “agree”, and keep a polarity flag so that the negation in “disagree” is not lost.

(b) Morphological Analysis

Word	Prefix	Root	Suffix	Transformation	Semantic effect
disagree	dis-	agree	-	Prefixation (Verb → Verb)	Negation: opposite of agree
agreement	-	agree	-ment	Suffixation (Verb → Noun)	State or result of agreeing
agreeable	-	agree	-able	Suffixation (Verb → Adjective)	Capable of or willing to agree

Morphological structure

- disagree = dis- + agree
- agreement = agree + -ment
- agreeable = agree + -able

(c) Python Program

```
# Q2: Morphological parser for sentiment analysis

ROOTS    = {"agree"}
PREFIXES = {"dis": ("dis-", "Negation: gives the opposite meaning of the root")}
SUFFIXES = {
        "ment": ("-ment", "Verb -> Noun",      "state/result of agreeing"),
        "able": ("-able", "Verb -> Adjective", "capable of / willing to agree"),
}

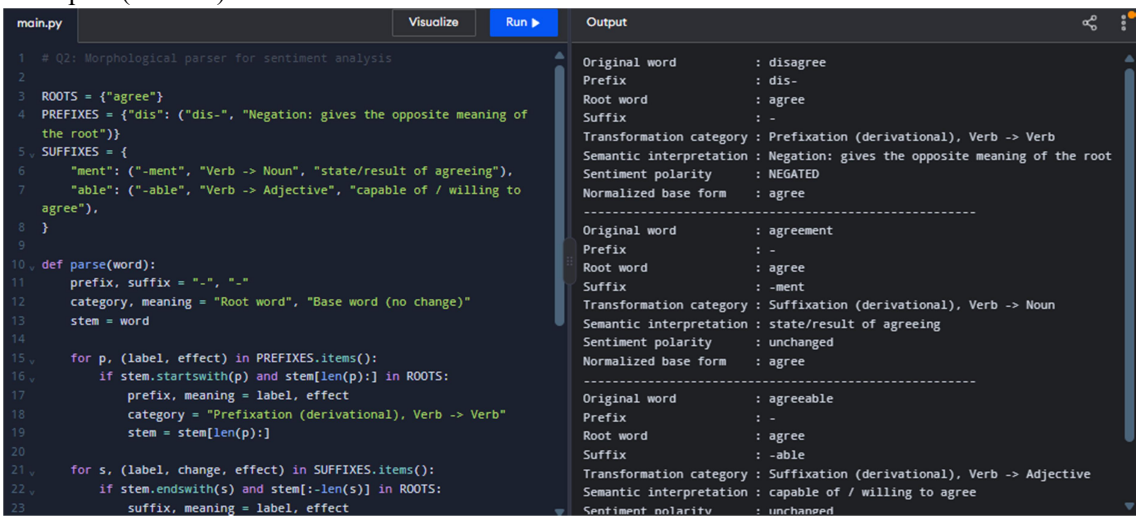
def parse(word):
        prefix, suffix = "-", "-"
        category, meaning = "Root word", "Base word (no change)"
        stem = word

        # Step 1: detect a prefix (the remaining part must be a known root)
        for p, (label, effect) in PREFIXES.items():
                if stem.startswith(p) and stem[len(p):] in ROOTS:
                        prefix, meaning = label, effect
                        category = "Prefixation (derivational), Verb -> Verb"
                        stem = stem[len(p):]

        # Step 2: detect a suffix (the remaining part must be a known root)
        for s, (label, change, effect) in SUFFIXES.items():
                if stem.endswith(s) and stem[:-len(s)] in ROOTS:
                        suffix, meaning = label, effect
                        category = "Suffixation (derivational), " + change
                        stem = stem[:-len(s)]
```

```
    return prefix, stem, suffix, category, meaning

for w in ["disagree", "agreement", "agreeable"]:
    prefix, root, suffix, category, meaning = parse(w)
    polarity = "NEGATED" if prefix == "dis-" else "unchanged"
    print(f"Original word          : {w}")
    print(f"Prefix                 : {prefix}")
    print(f"Root word              : {root}")
    print(f"Suffix                 : {suffix}")
    print(f"Transformation category  : {category}")
    print(f"Semantic interpretation  : {meaning}")
    print(f"Sentiment polarity       : {polarity}")
    print(f"Normalized base form     : {root}")
    print("-" * 55)
```



(d) Output

Original word : disagree
Prefix : dis-
Root word : agree
Suffix : -
Transformation category : Prefixation (derivational), Verb -> Verb
Semantic interpretation : Negation: gives the opposite meaning of the root
Sentiment polarity : NEGATED
Normalized base form : agree

Original word : agreement
Prefix : -
Root word : agree
Suffix : -ment
Transformation category : Suffixation (derivational), Verb -> Noun
Semantic interpretation : state/result of agreeing
Sentiment polarity : unchanged
Normalized base form : agree

Original word : agreeable

Prefix : -
Root word : agree
Suffix : -able
Transformation category : Suffixation (derivational), Verb -> Adjective
Semantic interpretation : capable of / willing to agree
Sentiment polarity : unchanged
Normalized base form : agree

Question 3: Morphology-Based Normalization for Topic Modeling

Scenario: A news aggregation pipeline contains "govern", "government" and "governance". They must be standardized for topic modeling and clustering.

(a) Approach

1. Keep the root "govern" in the lexicon.
2. Add the derivational suffix rules -ment and -ance (both Verb → Noun).
3. Strip the suffix and check that the remainder is the known root.
4. Count the derivational layers to get the derivational level (root = level 0).
5. Map all variants to the same base form "govern" for topic modeling.

(b) Morphological Analysis

Word	Root	Affix	Level	Hierarchy	Normalized
govern	govern	-	0	[govern]V	govern
government	govern	-ment	1	[[govern]V -ment]N	govern
governance	govern	-ance	1	[[govern]V -ance]N	govern

Morphological structure

- govern = root (level 0)
- government = govern + -ment (level 1)
- governance = govern + -ance (level 1)

(c) Python Program

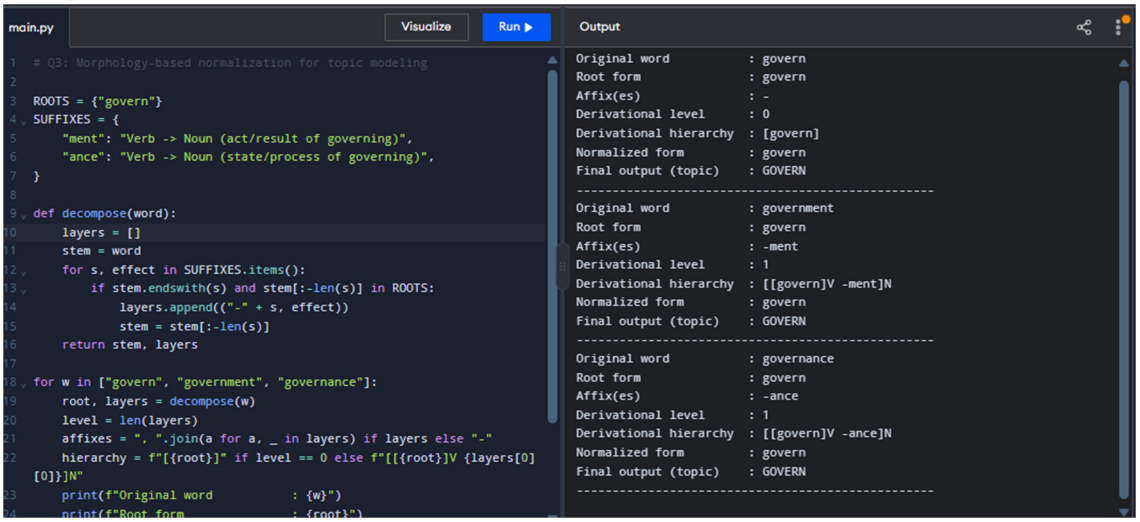
```
# Q3: Morphology-based normalization for topic modeling

ROOTS = {"govern"}
SUFFIXES = {
    "ment": "Verb -> Noun (act/result of governing)",
    "ance": "Verb -> Noun (state/process of governing)",
}

def decompose(word):
    layers = []                 # derivational layers found
    stem = word
```

```
        for s, effect in SUFFIXES.items():
            if stem.endswith(s) and stem[:-len(s)] in ROOTS:
                layers.append(("-" + s, effect))
                stem = stem[:-len(s)]
        return stem, layers

for w in ["govern", "government", "governance"]:
        root, layers = decompose(w)
        level = len(layers)
        affixes = ", ".join(a for a, _ in layers) if layers else "-"
        hierarchy = f"[{root}]" if level == 0 else f"[[{root}]V {layers[0][0]}]N"
        print(f"Original word          : {w}")
        print(f"Root form              : {root}")
        print(f"Affix(es)              : {affixes}")
        print(f"Derivational level     : {level}")
        print(f"Derivational hierarchy : {hierarchy}")
        print(f"Normalized form        : {root}")
        print(f"Final output (topic)   : {root.upper()}")
        print("-" * 50)
```



(d) Output

Original word : govern
Root form : govern
Affix(es) : -
Derivational level : 0
Derivational hierarchy : [govern]
Normalized form : govern
Final output (topic) : GOVERN

Original word : government
Root form : govern
Affix(es) : -ment
Derivational level : 1
Derivational hierarchy : [[govern]V -ment]N
Normalized form : govern

Final output (topic) : GOVERN

Original word : governance
Root form : govern
Affix(es) : -ance
Derivational level : 1
Derivational hierarchy : [[govern]V -ance]N
Normalized form : govern
Final output (topic) : GOVERN

Question 4: Derivational Parsing for Document Classification

Scenario: A document classifier receives “activate”, “activation” and “reactivation”. It must identify the derivational layers and map the words to one base form.

(a) Approach

1. Keep the valid base forms act, active and activate in the lexicon.
2. Add the prefix rule re- (again) and the suffix rules -ive, -ate and -ion (written -ation after -ate).
3. Peel one layer at a time: check the prefix first, then the suffix, until the root “act” is reached.
4. Reverse the peeled layers to get the derivational sequence from the root to the word.
5. Record the effect of each layer on word class and meaning, and map every word to the base form “activate”.

(b) Morphological Analysis

Word	Prefix	Root	Suffix	Derivational sequence
activate	-	act	-ive, -ate	act → active → activate
activation	-	act	-ive, -ate, -ion	act → active → activate → activation
reactivation	re-	act	-ive, -ate, -ion	act → active → activate → reactivate → reactivation

Morphological structure

- -ive: Verb → Adjective (having the quality of)
- -ate: Adjective → Verb (to make or cause to be)
- re-: Verb → Verb; the word class stays the same, only the meaning changes (again)
- -ion: Verb → Noun (act or result of the action)

(c) Python Program

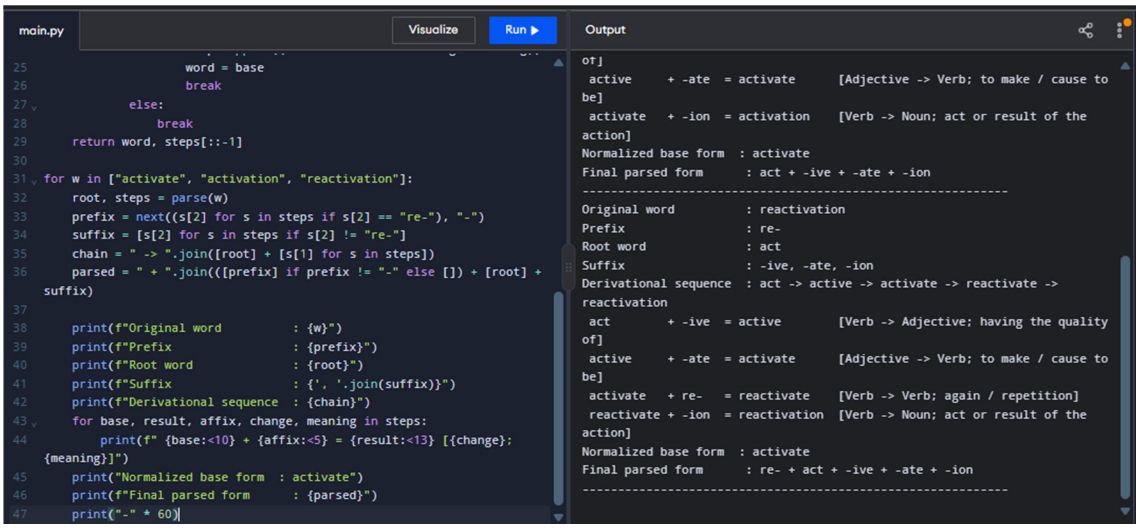
```
# Q4: Morphological parsing of activate / activation / reactivation

KNOWN    = {"act", "active", "activate"}      # valid base forms
```

```
PREFIXES = {"re": ("re-", "Verb -> Verb", "again / repetition")}
# (surface suffix, text restored, affix, class change, meaning)
SUFFIXES = [
        ("ation", "ate", "-ion", "Verb -> Noun",      "act or result of the action"),
        ("ate",   "e",   "-ate", "Adjective -> Verb", "to make / cause to be"),
        ("ive",   "",    "-ive", "Verb -> Adjective", "having the quality of"),
]

def parse(word):
        steps = []                    # each step: (from, to, affix, change, meaning)
        while word != "act":
                for p, (label, change, meaning) in PREFIXES.items():
                        if word.startswith(p) and word[len(p):] in KNOWN:
                                steps.append((word[len(p):], word, label, change, meaning))
                                word = word[len(p):]
                                break
                else:
                        for s, restore, affix, change, meaning in SUFFIXES:
                                if word.endswith(s):
                                        base = word[:-len(s)] + restore
                                        steps.append((base, word, affix, change, meaning))
                                        word = base
                                        break
                        else:
                                break               # no rule applies
        return word, steps[::-1]         # innermost derivation first

for w in ["activate", "activation", "reactivation"]:
        root, steps = parse(w)
        prefix = next((s[2] for s in steps if s[2] == "re-"), "-")
        suffix = [s[2] for s in steps if s[2] != "re-"]
        chain  = " -> ".join([root] + [s[1] for s in steps])
        parsed = " + ".join(([prefix] if prefix != "-" else []) + [root] + suffix)
        print(f"Original word        : {w}")
        print(f"Prefix               : {prefix}")
        print(f"Root word            : {root}")
        print(f"Suffix               : {', '.join(suffix)}")
        print(f"Derivational sequence : {chain}")
        for base, result, affix, change, meaning in steps:
                print(f"   {base:<10} + {affix:<5} = {result:<13} [{change}; {meaning}]")
        print(f"Normalized base form  : activate")
        print(f"Final parsed form     : {parsed}")
        print("-" * 60)
```



(d) Output

Original word : activate
Prefix : -
Root word : act
Suffix : -ive, -ate
Derivational sequence : act -> active -> activate
act + -ive = active [Verb -> Adjective; having the quality of]
active + -ate = activate [Adjective -> Verb; to make / cause to be]
Normalized base form : activate
Final parsed form : act + -ive + -ate

Original word : activation
Prefix : -
Root word : act
Suffix : -ive, -ate, -ion
Derivational sequence : act -> active -> activate -> activation
act + -ive = active [Verb -> Adjective; having the quality of]
active + -ate = activate [Adjective -> Verb; to make / cause to be]
activate + -ion = activation [Verb -> Noun; act or result of the action]
Normalized base form : activate
Final parsed form : act + -ive + -ate + -ion

Original word : reactivation
Prefix : re-
Root word : act
Suffix : -ive, -ate, -ion
Derivational sequence : act -> active -> activate -> reactivate -> reactivation
act + -ive = active [Verb -> Adjective; having the quality of]
active + -ate = activate [Adjective -> Verb; to make / cause to be]
activate + re- = reactivate [Verb -> Verb; again / repetition]
reactivate + -ion = reactivation [Verb -> Noun; act or result of the action]
Normalized base form : activate
Final parsed form : re- + act + -ive + -ate + -ion

Question 5: Inflectional Normalization for Search Optimization

Scenario: A search module receives “create”, “creates” and “creating”. All grammatical variants must be mapped to one base form without changing the meaning.

(a) Approach

1. Keep the root “create” in the lexicon.
2. Add the inflectional rules -s (third-person singular) and -ing (present participle, restore the final e).
3. Strip the suffix, restore the root and verify it in the lexicon.
4. Classify each word by its grammatical feature; all three are inflectional.
5. Return the base form “create” as the normalized output.

(b) Morphological Analysis

Word	Suffix	Grammatical category	Root	Normalized
create	-	Base form (present tense)	create	create
creates	-s	Third-person singular, present tense	create	create
creating	-ing	Present participle (progressive aspect)	create	create

Morphological structure

- create = base form
- creates = create + -s
- creating = creat(e) + -ing (the final e is dropped before -ing)

(c) Python Program

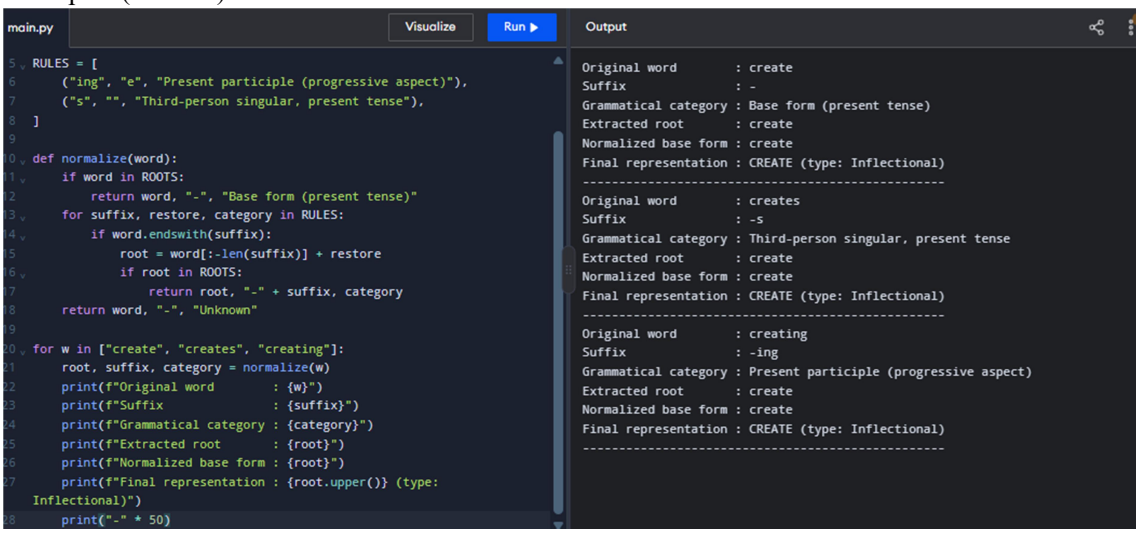
```
# Q5: Inflectional normalization for search optimization

ROOTS = {"create"}
# (suffix, text restored to root, grammatical category)
RULES = [
        ("ing", "e", "Present participle (progressive aspect)"),
        ("s",   "",  "Third-person singular, present tense"),
]

def normalize(word):
        if word in ROOTS:
                return word, "-", "Base form (present tense)"
        for suffix, restore, category in RULES:
                if word.endswith(suffix):
                        root = word[:-len(suffix)] + restore
                        if root in ROOTS:
```

```
                return root, "-" + suffix, category
    return word, "-", "Unknown"

for w in ["create", "creates", "creating"]:
    root, suffix, category = normalize(w)
    print(f"Original word      : {w}")
    print(f"Suffix             : {suffix}")
    print(f"Grammatical category: {category}")
    print(f"Extracted root     : {root}")
    print(f"Normalized base form: {root}")
    print(f"Final representation: {root.upper()}  (type: Inflectional)")
    print("-" * 50)
```



(d) Output

Original word : create
Suffix : -
Grammatical category: Base form (present tense)
Extracted root : create
Normalized base form: create
Final representation: CREATE (type: Inflectional)

Original word : creates
Suffix : -s
Grammatical category: Third-person singular, present tense
Extracted root : create
Normalized base form: create
Final representation: CREATE (type: Inflectional)

Original word : creating
Suffix : -ing
Grammatical category: Present participle (progressive aspect)
Extracted root : create
Normalized base form: create
Final representation: CREATE (type: Inflectional)